

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1003	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	K V CHARAN TEJA	Reg. No.:	192424167
Date:	04-08-2026	Duration:	60 minutes

Code of Conduct

I _ K V CHARAN TEJA _(192424167)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K V CHARAN TEJA

Annexure A – Architecture Design Assignment

ACADEMIC PROJECT, ALLOCATION MONITORING AND EVALUATION MANAGEMENT SYSTEM

- Project Title
 - Academic Project Allocation, Monitoring and Evaluation Management System
-

1. Analyze System Requirements

- 1.1 Study the Problem Statement and Identify the System Objectives
 - The Academic Project Allocation, Monitoring and Evaluation Management System is developed to automate the entire academic project management process in colleges and universities. In the existing system, projects are mainly allocated based on students' CGPA, which does not consider their interests, technical skills, or preferred domains. Manual monitoring and evaluation also consume significant time and may reduce transparency.
 - The proposed system provides an intelligent and digital platform that allocates projects according to students' interests and technical skills while enabling faculty members, project coordinators, HODs, administrators, and management to monitor project progress, evaluate students fairly, and maintain project records digitally.
 - System Objectives
 - Allocate projects based on students' interests and technical skills.
 - Improve project quality and practical learning.
 - Monitor project progress digitally.
 - Maintain project records securely.
 - Conduct transparent and fair evaluations.
 - Improve communication among stakeholders.
 - Reduce manual paperwork.
 - Generate reports automatically.
 - Increase productivity and efficiency.
-

1.2 Functional Requirements

- The proposed system shall provide the following functionalities:
-

• Requirement ID	• Functional Requirement
• FR1	• User Registration and Login
• FR2	• Student Profile Management
• FR3	• Interest and Skill Selection
• FR4	• Project Allocation

• Requirement ID	• Functional Requirement
• FR5	• Faculty Guide Assignment
• FR6	• Project Progress Tracking
• FR7	• Internal Evaluation
• FR8	• External Evaluation
• FR9	• Feedback Management
• FR10	• Report Generation
• FR11	• User and Project Management
• FR12	• Notification System
• FR13	• Dashboard for all stakeholders
• FR14	• Secure Data Management

1.3 Non-Functional Requirements

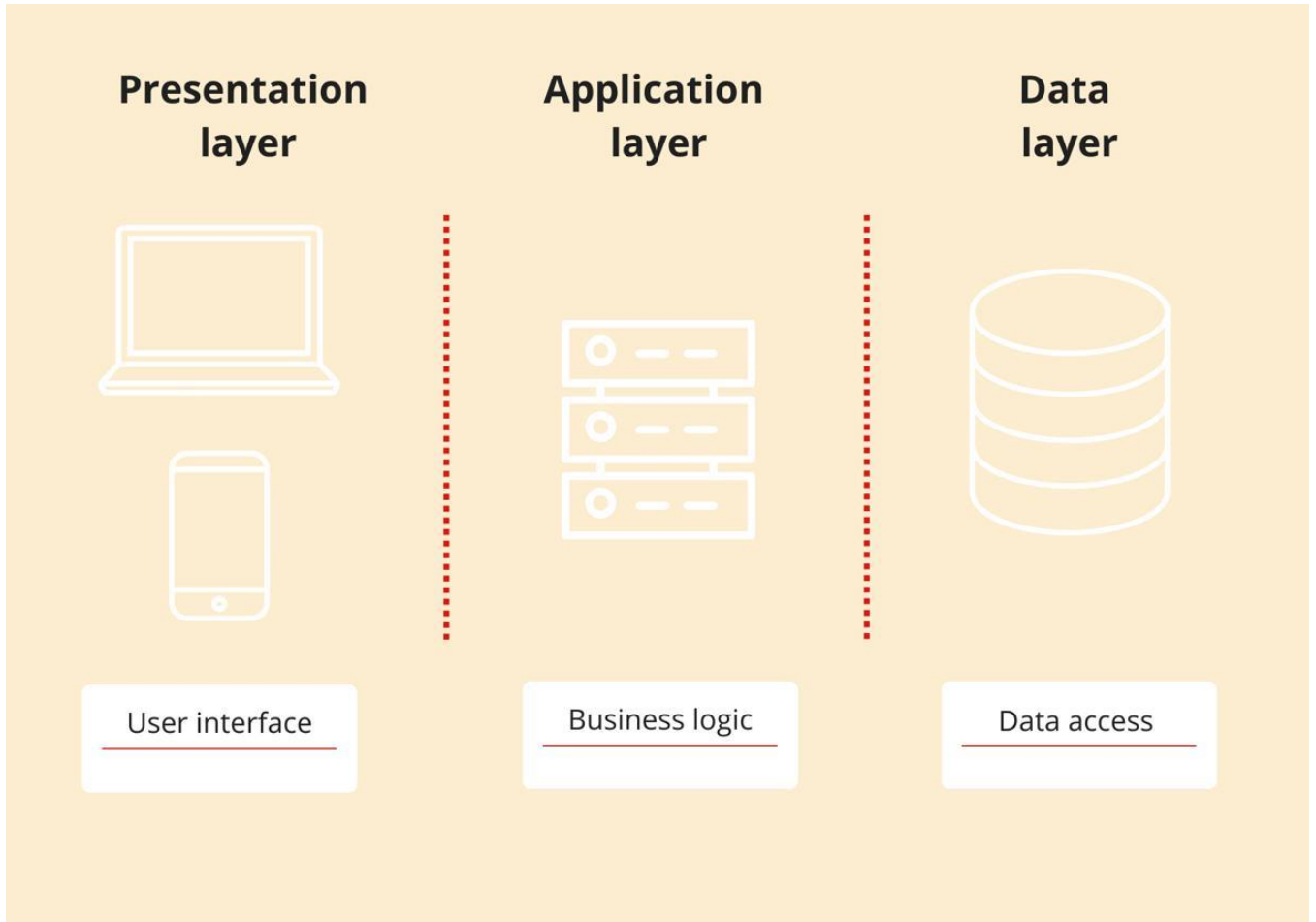
- Performance
 - Fast response time.
 - Support multiple concurrent users.
 - Efficient database operations.
 - Security
 - Secure Login.
 - Password Encryption.
 - Role-Based Access Control.
 - Secure Database Storage.
 - Reliability
 - Accurate project allocation.
 - Minimum failures.
 - Consistent database operations.
 - Usability
 - Easy-to-use interface.
 - Responsive design.
 - Simple navigation.
 - Scalability
 - Support increasing students.
 - Support additional departments.
 - Support more faculty members.
 - Maintainability
-

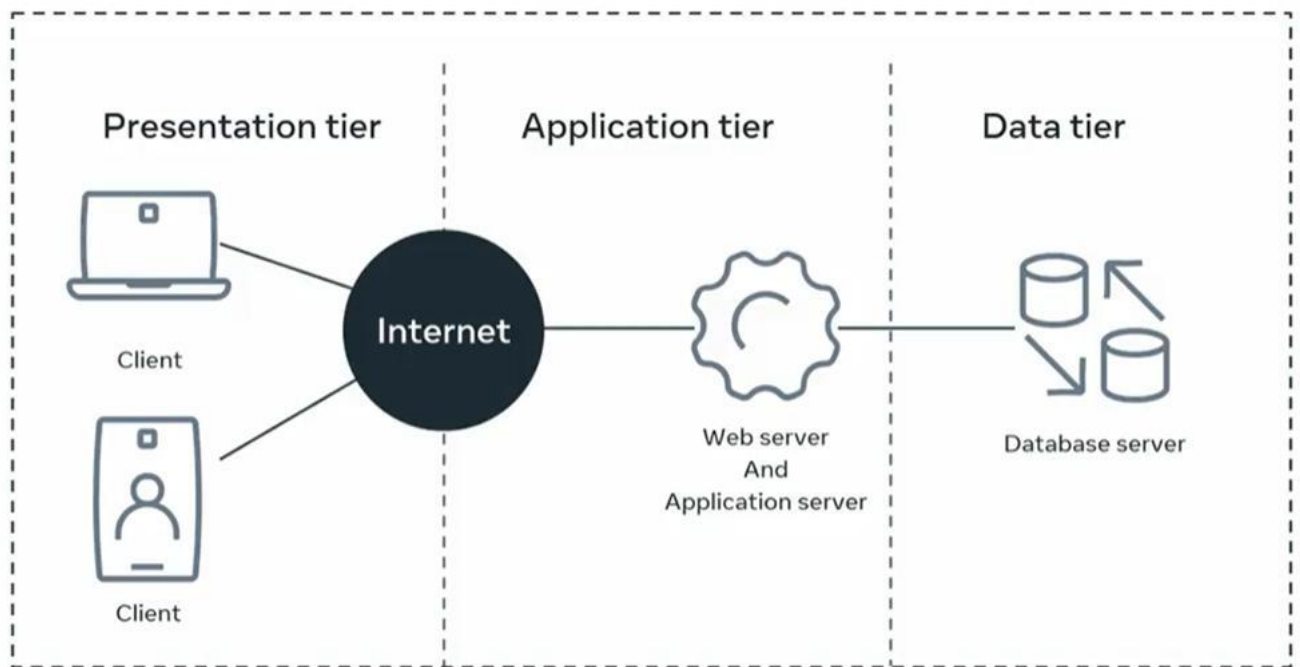
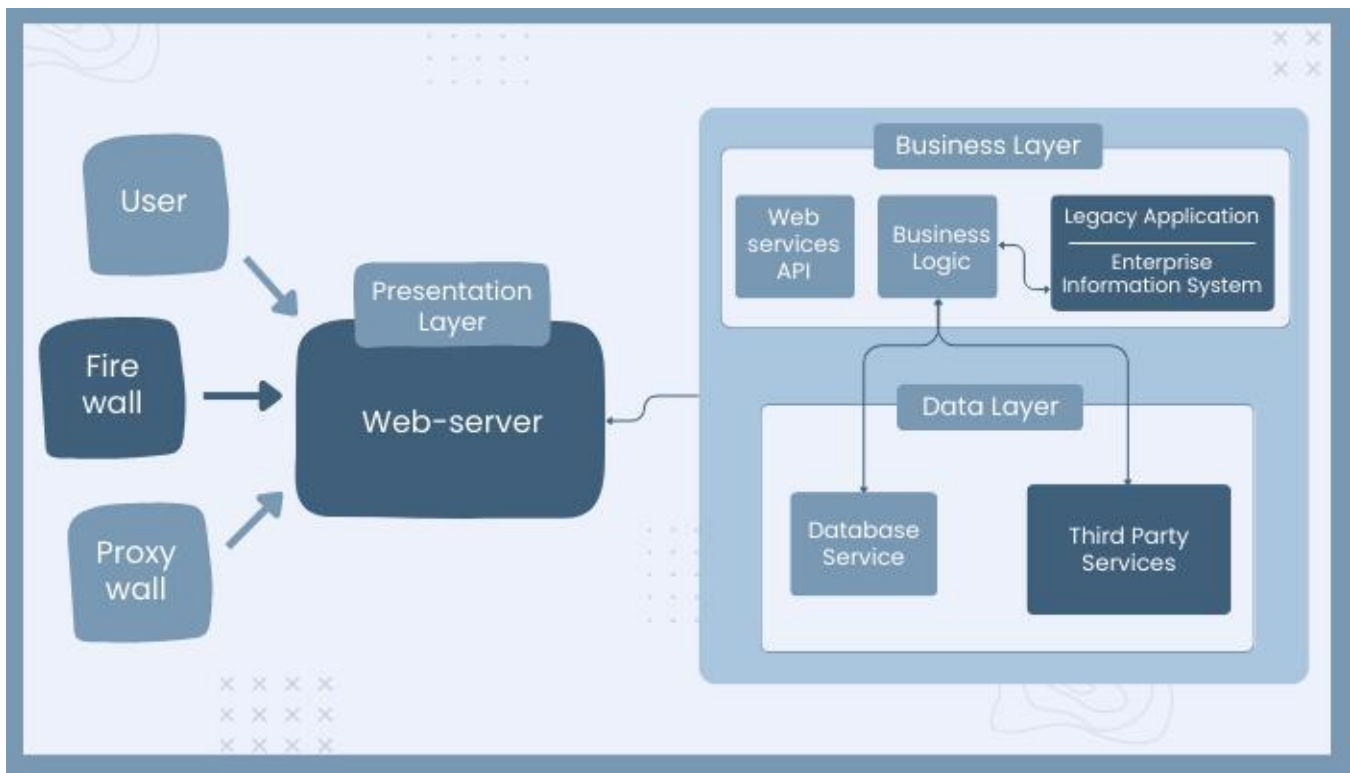
- Modular software design.
- Easy updates.
- Easy debugging.
- Availability
- 24×7 accessibility.
- Minimum downtime.
- Backup and Recovery
- Automatic database backup.
- Disaster recovery mechanism.

1.4 Quality Attributes

- Scalability
 - The architecture should support thousands of students, faculty members, departments, and academic projects without affecting performance.
 - Maintainability
 - Each module should be independent so developers can update or modify the software without affecting the remaining modules.
 - Reliability
 - The application should always provide accurate project information with minimum system failures.
 - Availability
 - The application should be accessible throughout the project lifecycle.
 - Performance
 - The application should provide quick response during login, project allocation, report generation, and evaluation.
 - Security
 - The system should implement authentication, authorization, password encryption, secure APIs, and role-based access control.
-

Architecture Overview





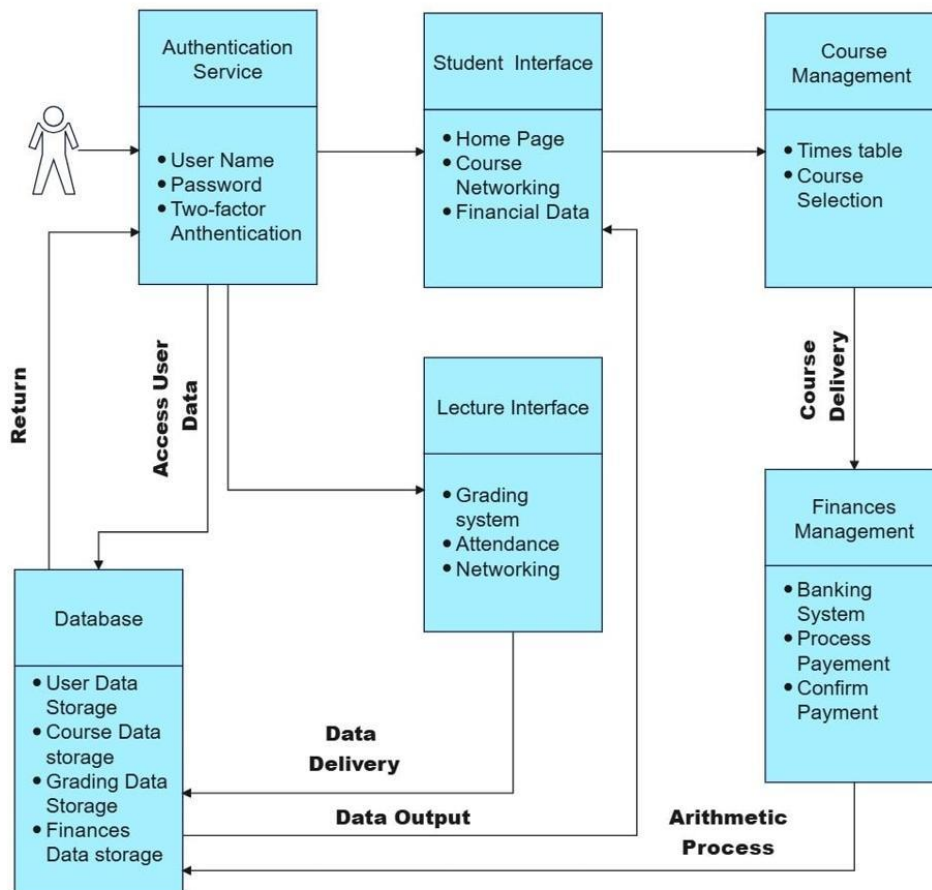
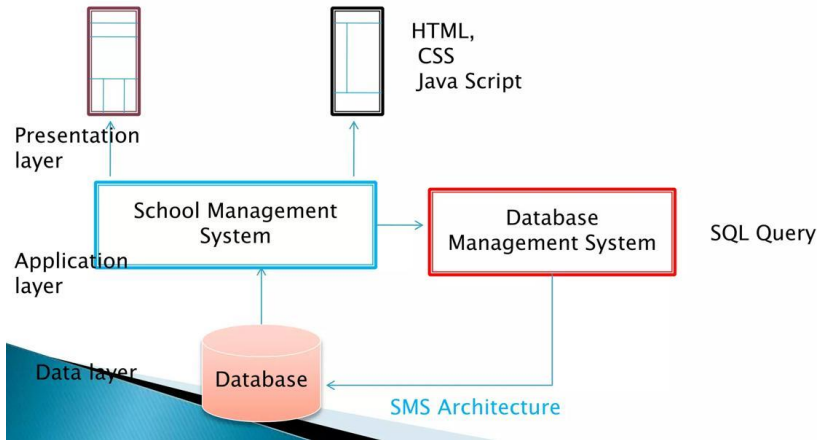
- 2. Select a Suitable Software Architecture
- Selected Architecture
- Layered Architecture (Three-Tier Architecture)
- The proposed system follows the Layered Architecture, which separates the application into three independent layers.
- Presentation Layer

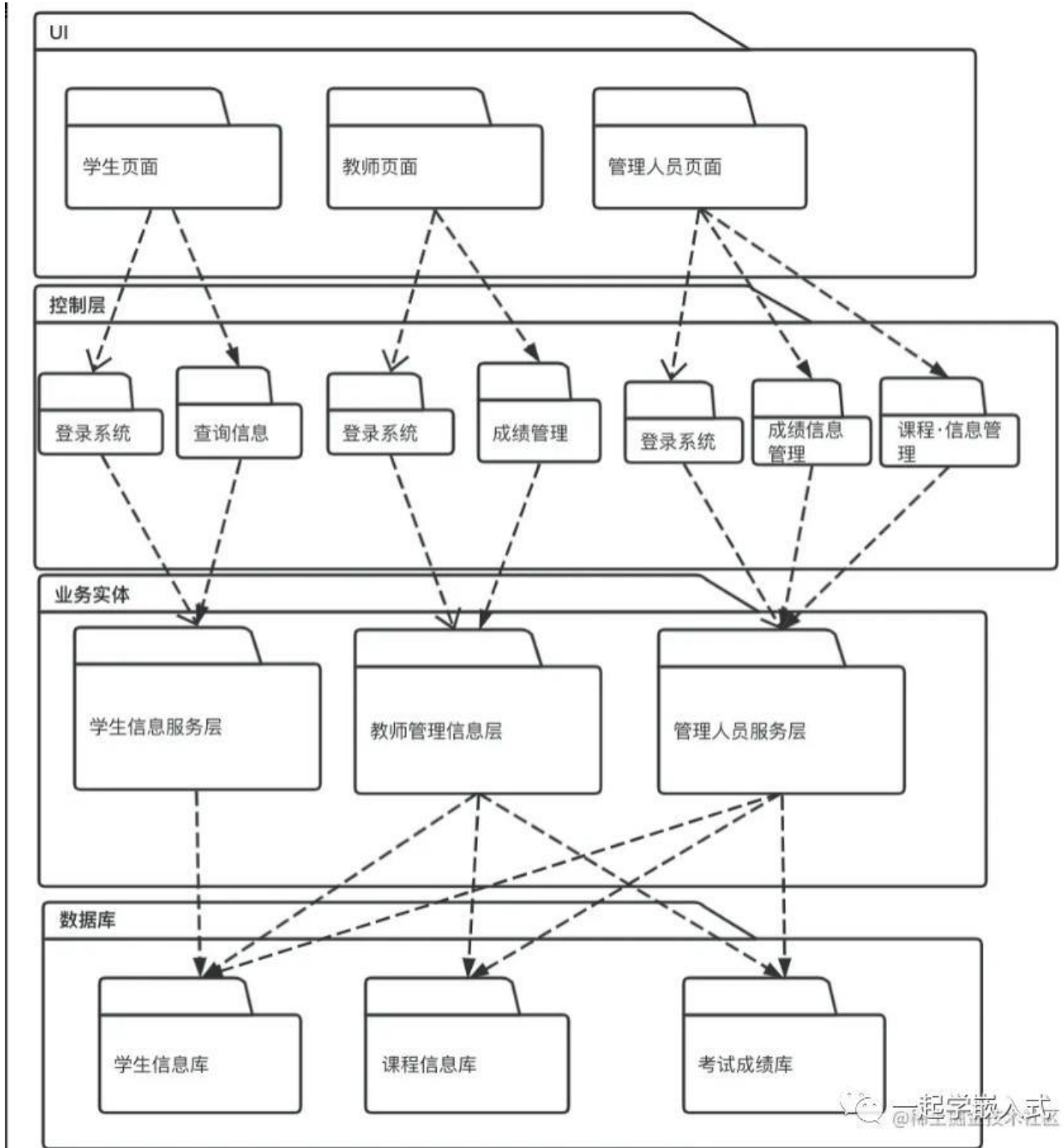
- Provides the graphical interface for:
 - Students
 - Faculty
 - HOD
 - Project Coordinator
 - Administrator
 - External Examiner
 - Business Logic Layer
 - Processes
 - Authentication
 - Project Allocation
 - Evaluation
 - Notifications
 - Report Generation
 - Faculty Assignment
 - Data Layer
 - Stores
 - Student Details
 - Faculty Details
 - Project Details
 - Evaluation Records
 - Reports
 - User Accounts
 - ---
 - Justification
 - Layered Architecture is selected because
 - Easy to understand.
 - Easy to develop.
 - Highly scalable.
 - Easy maintenance.
 - Better security.
 - Independent modules.
 - Supports future expansion.
 - Better performance.
 - Faster debugging.
 - Suitable for full-stack web applications.
-

3. Software Architecture Diagram

System Architecture

System design is the process of defining the architecture, components, modules, interfaces and data for a system to satisfy special requirements.





Major Components

- Presentation Layer
- Login
- Registration
- Student Dashboard
- Faculty Dashboard
- HOD Dashboard

- Coordinator Dashboard
- Administrator Dashboard

•

- Business Logic Layer
- Authentication Module
- Student Management
- Project Allocation
- Faculty Assignment
- Evaluation Module
- Progress Tracking
- Report Generation
- Feedback Management
- Notification Service

•

- Database Layer
- Student Table
- Faculty Table
- Project Table
- Allocation Table
- Progress Report Table
- Evaluation Table
- Feedback Table
- User Account Table

•

- External Systems
- Email API
- SMS Gateway
- Cloud Storage
- File Upload Service

•

- Communication Mechanism
- HTTP / HTTPS
- REST API
- JDBC / SQL
- SMTP
- Cloud API

•

- 4. Explain Components and Their Interactions
- Presentation Layer

- Acts as the user interface where students, faculty members, HODs, administrators, and project coordinators interact with the system.
 - Responsibilities
 - Login
 - Registration
 - Dashboard
 - Upload Reports
 - View Allocation
 - View Marks
 - ---
 - Business Logic Layer
 - Responsible for implementing business rules.
 - Responsibilities
 - Authentication
 - Project Allocation
 - Faculty Assignment
 - Evaluation
 - Progress Monitoring
 - Report Generation
 - Notifications
 - ---
 - Database Layer
 - Stores all project-related information securely.
 - Responsibilities
 - Student Records
 - Faculty Records
 - Project Records
 - Evaluation Records
 - Feedback
 - User Accounts
 - ---
 - External Systems
 - Used for
 - Email Notifications
 - SMS Alerts
 - Cloud File Storage
 - ---
 - Component Communication
 - User accesses the web application.
-

- Request reaches Presentation Layer.
- Presentation Layer sends request to Business Logic Layer.
- Business Logic validates request.
- SQL query is sent to Database Layer.
- Database returns requested information.
- Business Logic processes data.
- Result is displayed on the Presentation Layer.
- Email/SMS notifications are sent when required.

-
- Overall Workflow
 - Student Registration
 - ↓
 - Profile Creation
 - ↓
 - Interest Selection
 - ↓
 - Project Allocation
 - ↓
 - Faculty Assignment
 - ↓
 - Project Development
 - ↓
 - Weekly Progress Upload
 - ↓
 - Faculty Monitoring
 - ↓
 - Internal Evaluation
 - ↓
 - External Evaluation
 - ↓
 - Feedback
 - ↓
 - Report Generation
 - ↓
 - Project Completion
-
- 5. Discuss Quality Attributes
 - Scalability
-

- Supports increasing users, projects, and departments while maintaining performance.

-

- Security
- Implements
- Role-Based Access Control
- Password Encryption
- Secure Login
- Data Encryption
- Secure APIs

-

- Performance
- Provides
- Fast Login
- Quick Project Allocation
- Faster Report Generation
- Efficient Database Queries

-

- Reliability
- Ensures
- Accurate Data
- Minimum Errors
- Data Consistency
- Automatic Backup

-

- Maintainability
- The layered architecture enables independent updates, debugging, testing, and feature enhancements without affecting other modules.

-

- Availability
- The application is available anytime with proper server management, backup, and monitoring.

-

- 6. Justify Architectural Decisions

- Rationale
- Layered Architecture is selected because it separates the system into Presentation, Business Logic, and Database layers. This separation improves software quality, simplifies development, reduces complexity, and enhances maintainability. It also allows multiple stakeholders to access the system securely while sharing a centralized database.

-

- Trade-offs
 - Advantages
-

- Easy Development
 - Easy Maintenance
 - Better Security
 - High Scalability
 - Better Code Reusability
 - Easier Testing
 - Better Reliability
 - Limitations
 - Slight increase in response time due to multiple layers.
 - Higher initial development effort.
 - More modules compared to a simple application.
 - Large-scale deployments may require additional optimization.
 - ---
 - Future Enhancements
 - The selected architecture supports future improvements such as:
 - AI-based Project Recommendation
 - Online Viva Management
 - Plagiarism Detection
 - Mobile Application
 - Cloud Deployment
 - LMS Integration
 - Email and SMS Automation
 - Analytics Dashboard
 - AI-based Student Performance Prediction
 - Third-party API Integration
-

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25